Systèmes interconnectés – SINT
Sujet de TP 5 : CAN et FPGA

Centrale Nantes
P.-E. Hladik, pehladik@ec-nantes.fr
—
Version bêta (26 novembre 2024)

1 Objectifs pédagogiques

— spécifier une machine séquentielle à l'aide d'un automate
— simuler et tester une implémentation en VHDL
— intégrer plusieurs entités dans un projet

2 Rendu

À la fin du TP déposez sur Hippocampus une archive avec un compte rendu en pdf (voir les éléments demandés dans le texte).

3 Prise en main

Cette première partie a pour but de (re)prendre en main Vivado et le VHDL.

3.1 Création d'un projet

Commencez par créer un nouveau projet sous Vivado (voir le document et les vidéos en ligne). Pour cette première étape vous aurez besoin des fichiers `clock_div.vhd` en tant que *Design Sources*, de `testbench_div.vhd` comme *Simulation Sources* et `basys3.xdc` comme contraintes. Ces fichiers sont disponibles dans le dépôt. Vous pouvez importer les fichiers à la création du projet ou bien après.

Pour la carte Basys3, il faut sélectionner le composant « xc7a35tcpg236-1 » que l'on peut trouver avec le filtre :
— Family : Artix-7
— Package : cpg236
— Speed : -1

3.2 Simulation

Vérifiez que vous avec bien importez `testbench_div.vhd` dans *Simulation Sources* et qu'il est bien au *top level* (sinon clic droit et *Set as Top*). Ce fichier simule le comportement d'une entité de type `clock_div`.

Lancez la simulation comportementale (voir vidéo) et observez. Pour l'instant seul le signal `reset` est simulé. Ajoutez le code nécessaire pour simuler l'horloge à une période de 10 ns et lancez la simulation.

Rapport : mettez l'extrait de code du testbench ainsi qu'une capture de la simulation.

Quel est le rapport cyclique si le diviseur est impair (3 par exemple) ? Observez en simulation le comportement et expliquez les raison en allant lire le code de `clock_div.vhd`.

Rapport : mettez une capture de la simulation avec un diviseur de 3 et expliquez le comportement observé à partir du code.

3.3 Synthèse

Assurez-vous que vous avez bien importé le fichier `basys3.xdc` avec les contraintes.

Créez un nouveau fichier de conception et mettez le comme *Top Level*. Implémentez un système qui fait clignoter une led toutes les 500 ms. Pour cela **utilisez l'entité diviseur d'horloge** vu en simulation sachant que l'horloge du système a une période de 10 ns. Les signaux physiques dont vous aurez besoin sont :

- `led0` en sortie,
- `reset` en entrée,
- `clock_100Mhz` en entrée.

Avant de faire l'implémentation écrivez l'automate de ce composant.

Les ports doivent être déclarés comme ports de votre entité comme dans l'exemple ci-dessous :

```
entity blink is
  Port (
        reset           : in std_logic;
        clock_100Mhz    : in std_logic;
        led0            : out std_logic
        );
end blink;
```

et seront ensuite liés aux signaux physiques dans le fichiers de contraintes (vérifiez que les lignes nécessaires dans le fichiers de contraintes `basys.xdc` sont bien décommentées).

Ecrivez le code, faites la synthèse et exécutez le résultat sur la carte.

Rapport : mettez l'automate de votre composant blink ainsi que le code produit.

4 Contrôleur CAN

L'objectif est de concevoir un contrôleur CAN sur le FPGA pour envoyer un message fixe avec un octet de donnée. L'identifiant sera fixé ainsi que la fréquence d'envoi (125000 Hz). La trame sera donc fixe pour la partie en-tête, mais changera pour la partie donnée. Il faudra donc recalculer le CRC.

Pour réaliser cela il faut mettre au point plusieurs dispositifs :

1. émettre la trame
2. gérer le bit-stuffing
3. calculer un CRC
4. détecter que la ligne est disponible pour émettre
5. gérer les priorités
6. gérer les erreurs (non acquitement)

Pour les derniers points il faut donc aussi écouter la ligne Rx.

Pour chacun de ces composants, décrivez **leur interface ainsi que l'automate de leur comportement**. Tester les en simulation puis en réelle.

Pour tester en réel il vous faudra un *transceiver*, une breadboard et quelques fils. Vous allez utiliser un MCP2562. La documentation est fournie avec le sujet. La communication entre le CAN et le transceiver se fera sur une broche d'un PMOD de la Basys 3 (à vous de la définir). Le composant MCP2562 ayant besoin d'une alimentation en 5V (et la sortie sur les PMOD étant à 3.3V) vous pouvez repiquer sur la breadboard l'USB qui sort de la carte en utilisanr l'adaptateur micro USB qui vous sera fourni (voir figure **??**).

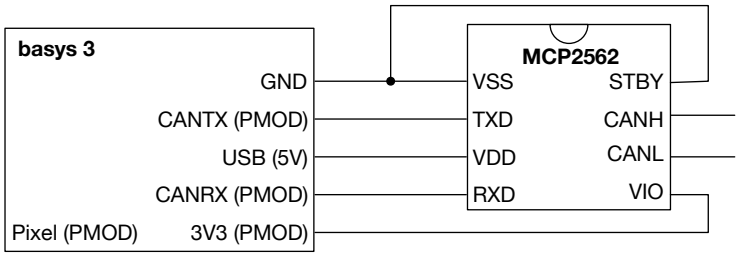


Fig. 1 – Schéma de connexion entre le basys 3 et le MCP2562

Rapport : pour chacune des sous-sections suivantes mettez dans votre rapport l'automate ainsi que le code produit et les résultats de tests réalisés. Pas d'automate, pas d'évaluation...

4.1 Test d'envoi d'une trame statique

Pour faire les choses progressivement, commencer par émettre une trame statique vers un récepteur CAN (Feather ou Teensys) et vérifier que la trame est bien reçue. Vous pouvez aussi utiliser l'analyseur logique pour vérifier que votre trame est correcte.

La trame de la figure **??** est tirée de https://en.wikipedia.org/wiki/CAN_bus (section bit stuffing) et représente la séquence de bits émis pour un message avec un id 0x14 avec une donnée d'un octet de valeur 0x01. La séquence comportement les bits dus au bit stuffing.

La trame a la valeur suivante : 0 0 0 0 0 1 0 0 1 0 1 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 1 0 0 1 0 1 0 0 0 0 1 1 0 0 0 0 0 1 0 0 1 0 1 1 1 1 1 1 1 1 1 1

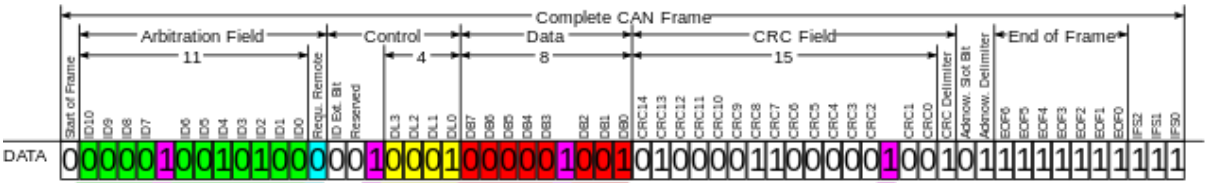


Fig. 2 – Exemple de trame émise (source https://en.wikipedia.org/wiki/CAN_bus)

4.2 Bit-stuffing

Modifier l'envoi pour gérer automatiquement le bit-stuffing, c'est-à-dire qu'il faut calculer à la volée les bits à ajouter en fonction de la trame envoyée. Tester toujours par une émission vers un récepteur ou l'analyseur logique.

4.3 Détection de l'état de la ligne

Réaliser un composant pour détecter que la ligne est libre. Pour cela, connecter votre dispositif à un émetteur CAN qui envoi périodiquement des messages. Sortez sur un port PMOD un signal indiquant si la ligne est libre ou non et vérifier avec l'analyseur que le comportement est correct.

4.4 Composition émission et état de la ligne

Composez vos modules d'émission avec le module de détection d'état de la ligne. Tester en interconnectant votre système avec un autre émetteur qui envoie périodiquement des messages et vérifier que votre émetteur n'émet pas quand la ligne n'est pas disponible.

4.5 Calcul du CRC

Passer ensuite au calcul du CRC avec une trame dont la partie donnée est variable et émettez le résultat. Utiliser l'analyseur logique pour vérifier que votre CRC est bon.

4.6 Gestion des priorités

Réaliser un composant qui vérifie la priorité en cas de collision. Pour tester votre dispositif, composer un système sur el FPGA avec deux émetteurs synchrones et sortez une ligne pour indiquer qui est prioritaire. Vérifiez à l'aide de l'analyseur logique que tout se passe bien.

4.7 Gérer les erreurs

Si vous en êtes là c'est qu'a priori vous êtes aptes à imaginer comment ajouter et tester ce composant...